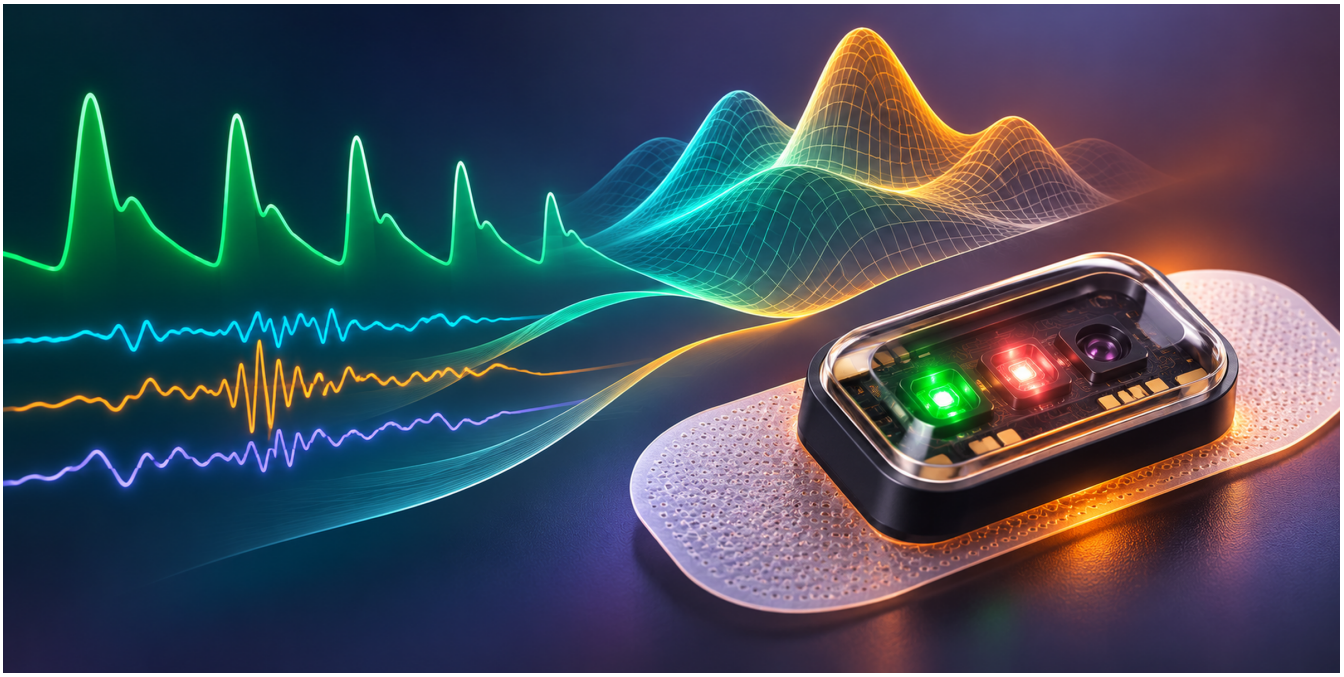
Wearable Signal Scoring Expressions with paceval

Industrial IMU / PPG feature processing and versioned mathematical scores



Concept illustration, not a computed scientific result. All scientific result figures use actual paceval output.

Executed research outcome

An original paceval C++ integration evaluated 51 persistent mathematical graphs using features from 10,000 deterministic synthetic IMU/PPG windows. All 50 scoring formulas, plus one revision, were compiled into reusable paceval handles and independently checked.

Synthetic windows	10,000 windows; 2,000,000 raw timestamps
Mathematical execution	13,923 scalar values; 1,600 batch values
Independent high-precision check	500 selected scores passed 80-digit verification
Scientific visualizations	Eight charts based on recorded paceval outputs

Scope: Non-clinical research demonstration only. No real people, diagnostic alerts, regulated device claims or clinical efficacy results are part of this experiment.

1. Industrial use case and current state of the art

A modern wrist-worn device often combines optical photoplethysmography (PPG) with an inertial measurement unit (IMU). The optical signal contains pulsatile modulation but can be corrupted by motion, device contact, environmental light and sensor characteristics. Movement and optical signals therefore need separate preprocessing before a configurable scoring stage.

Practical engineering workflow

Sensor acquisition leads to short time windows; host preprocessing computes standardized optical and motion features; a mathematical evaluation engine assigns configurable scores; and a host decision system presents or stores the outputs. paceval is tested here only for the mathematical scoring engine.

Contemporary research and published resources

A 2026 technical review summarizes wearable ECG/PPG fusion, motion artifacts and signal-quality indices [1]. A 2025 PPG signal-quality review and validation study examines motion-related uncertainty in optical cardiovascular monitoring [2]. PPG-DaLiA provides recorded motion and physiological data from 15 participants [3], while PhysioNet provides additional exercise-based PPG and motion records [4]. None of these real-world datasets is used as input in this demonstration.

Research question and contribution

Can 50 mathematically different wearable feature-to-score rules be created once, evaluated repeatedly from fixed synthetic signals and revised without overwriting earlier expressions? The experiment verifies source-level reproducibility and the mathematics of an explicit postprocessing layer; it does not validate a pulse detector, clinical classifier, or embedded product.

Regulatory framing

The FDA general-wellness guidance updated in January 2026 distinguishes certain low-risk wellness functions from disease-oriented medical-device purposes [5]. This prototype makes no health diagnosis, risk prediction, treatment or clearance claim.

2. Defined synthetic signals and preprocessing

Every synthetic window spans 8 seconds at 25 Hz (200 sample times). A deterministic C++ generator combines a periodic optical component with motion-dependent contamination and three-axis acceleration around a constant gravity term. An independent host procedure extracts six window-level features.

Raw signal construction and physical assumptions

The PPG channel is a normalized optical proxy, not an optical measurement in volts or an actual arterial waveform. Acceleration channels are normalized in units of g. Fixed seeds and bounded rates allow each window to be reconstructed. The generated pulse oscillation and movement cadence are not observed clinical measurements.

$$A_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(a_{x,i}^2 + a_{y,i}^2 + (a_{z,i}-1)^2\right)}$$

Here N = 200. The host additionally counts upward acceleration zero crossings as a cadence proxy and searches positive optical autocorrelation peaks at lags 12 through 38. The selected lag provides an illustrative pulse-frequency proxy; this is not a validated estimation algorithm.

Six dimensionless features supplied to paceval

a	Acceleration RMS divided by 0.65 g
c	Cadence proxy divided by 145 events per minute
h	Autocorrelation pulse proxy divided by 140 bpm
m	Motion artifact proxy divided by 0.65
p	PPG standard deviation / mean, divided by 0.12
q	Normalized optical autocorrelation peak, clipped to [0,1]

An input contract rejects nonfinite values and values outside the demonstration feature domain. The physical and statistical meaning of these normalization constants is intentionally limited to this synthetic workload.

3. Mathematical scoring functions

Every score is a monotone nonlinear transformation of a dimensionless model expression. Five formula families each use ten distinct coefficient pairs, creating 50 immutable formula descriptions.

$$S(z) = \frac{1}{1 + \exp(-z)}$$

$$z_1 = \alpha a + \beta c + 0.14h - 0.37$$

$$z_2 = \alpha q - \beta m + 0.27p - 0.30$$

$$z_3 = \alpha ac + \beta pq - 0.28m - 0.37$$

$$z_4 = \alpha \sin(a) + \beta \cos(h) + 0.32q - 0.42m - 0.43$$

$$z_5 = \alpha \ln(1 + a^2) + \beta \exp(-m^2) + 0.19pq - 0.66$$

$$\alpha = 0.36 + 0.049j + 0.031f, \quad \beta = 0.22 + 0.035j$$

Family f = 0..4 and variant j = 0..9. The expressions test additive, multiplicative, trigonometric, logarithmic and exponential operators. The formula semantics are explicit in src/wearable_scoring.cpp and saved unmodified as CSV strings in examples/formula_registry.csv.

Interpretation of the mathematical outputs

A bounded 0-to-1 score is not a calibrated clinical probability. These coefficients were chosen for a transparent technical demonstration, not fitted to physiological labels, activity categories, adverse events or clinical reference measurements.

4. Original paceval execution and workload

The application compiles the original paceval_main.cpp SDK bridge together with the C++ application, and loads the compatible external Linux paceval runtime. Formula strings are registered once with the original API; handles persist throughout the replay and are deleted at program shutdown.

Initializes runtime	paceval_InitializeLibrary
Compiles 51 expressions	paceval_CreateComputation
Scalar evaluation	paceval_dGetComputationResult
Independent-vector batch evaluation	paceval_dGetComputationResultExt
Error and validity check	paceval_GetIsError; per-result batch error codes
Releases graph handles	paceval_DeleteComputation and paceval_FreeLibrary

Executed synthetic workload

Raw timestamps	2,000,000 (10,000 windows x 200 samples)
Different scoring formulas	50 original plus one separate revision
Scalar score computations	13,923
Batch results	1,600 (50 formulas x 32 input vectors)
Computed score-surface values	3,721 (61 x 61 parameter samples)
Synthetic threshold flags	420 of 10,000 at score >= 0.65

Host tasks: generate and preprocess raw synthetic signals, enforce domains, control thresholds, serialize files and independently validate the outputs. paceval tasks: parse and evaluate the defined mathematical expressions. Only the latter tasks are attributed to paceval.

5. Actually generated IMU/PPG traces and replay

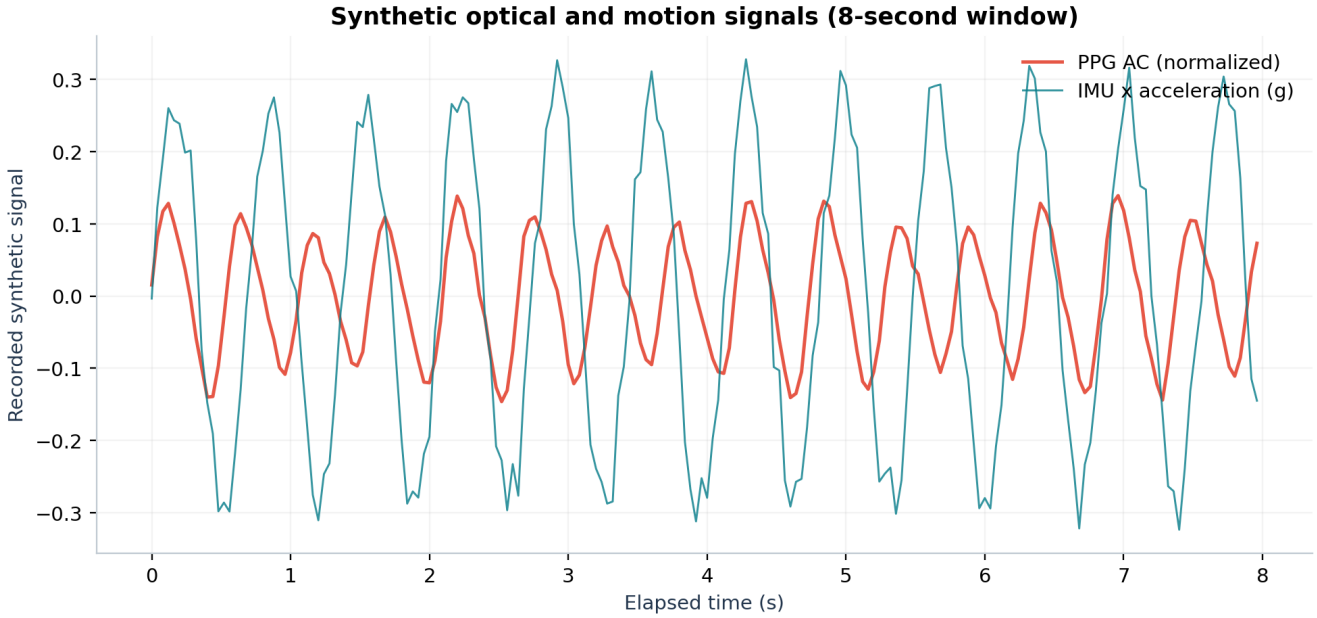


Figure 1. A saved synthetic optical and accelerometer window. The curves are signals generated in C++, not patient or device recordings.

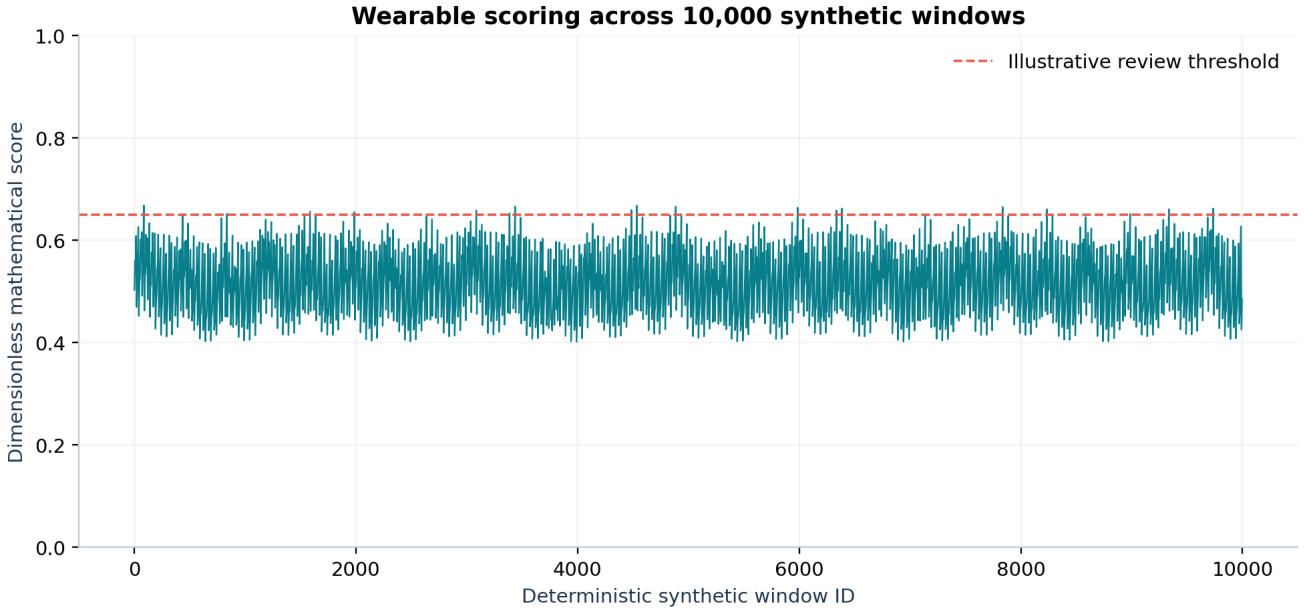


Figure 2. Score replay from 10,000 actual paceval evaluations; a fixed synthetic technical review threshold is marked.

6. Score distributions and operating policy

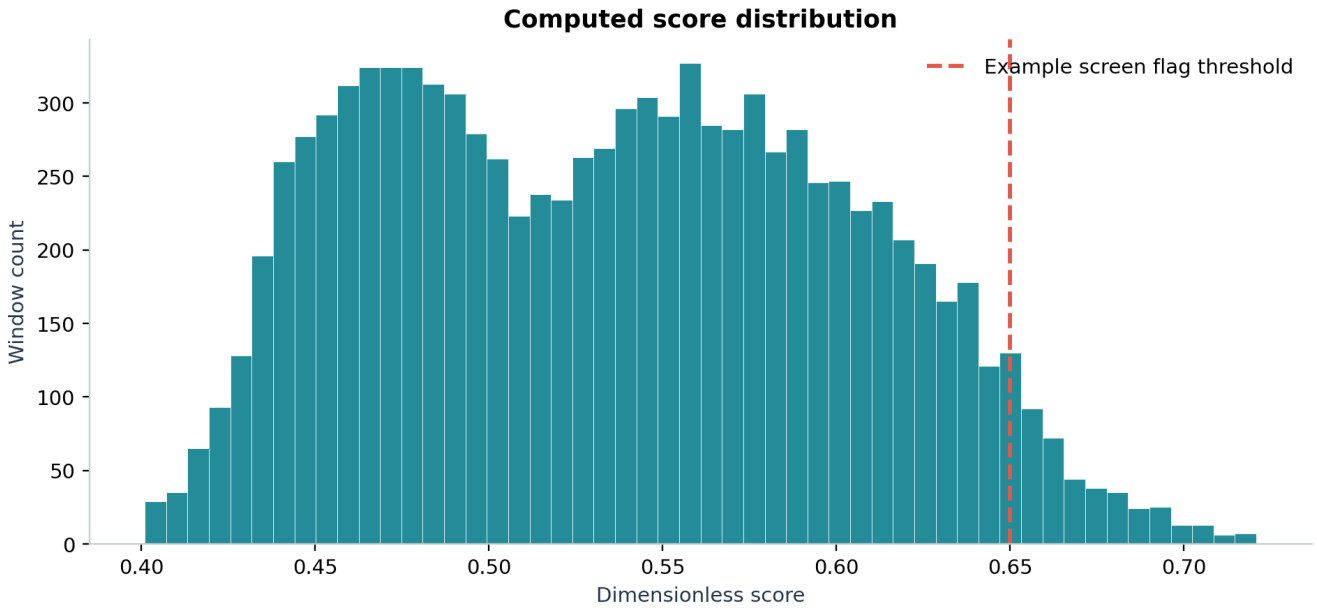


Figure 3. Distribution of recorded mathematical scores over the 10,000 synthetic windows.

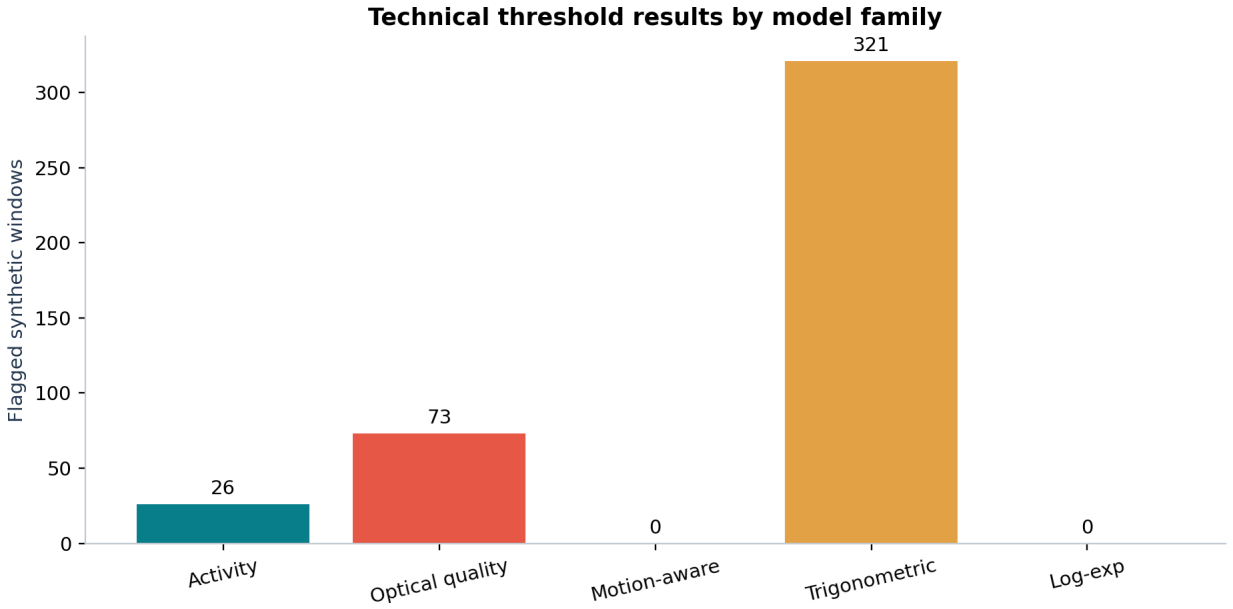


Figure 4. The selected mathematical expression family changes the number of technical screen flags.

What the example policy means

The fixed cutoff of 0.65 is defined by the external C++ host. Its 420 triggered flags are neither diagnoses nor estimates of medical sensitivity, specificity, risk, clinical prevalence or real-device alert performance.

7. Multidimensional signal-quality sensitivity

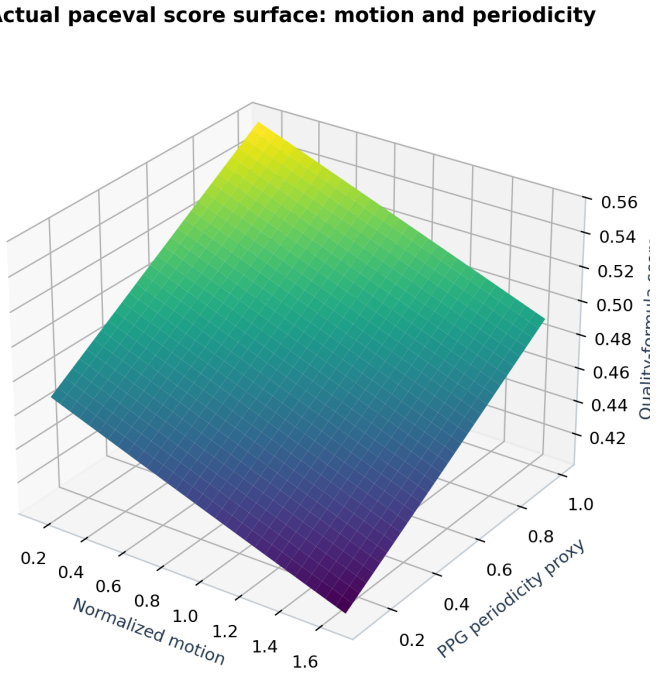


Figure 5. A real 61 x 61 paceval score field over normalized motion and optical periodicity features (fixed remaining inputs).

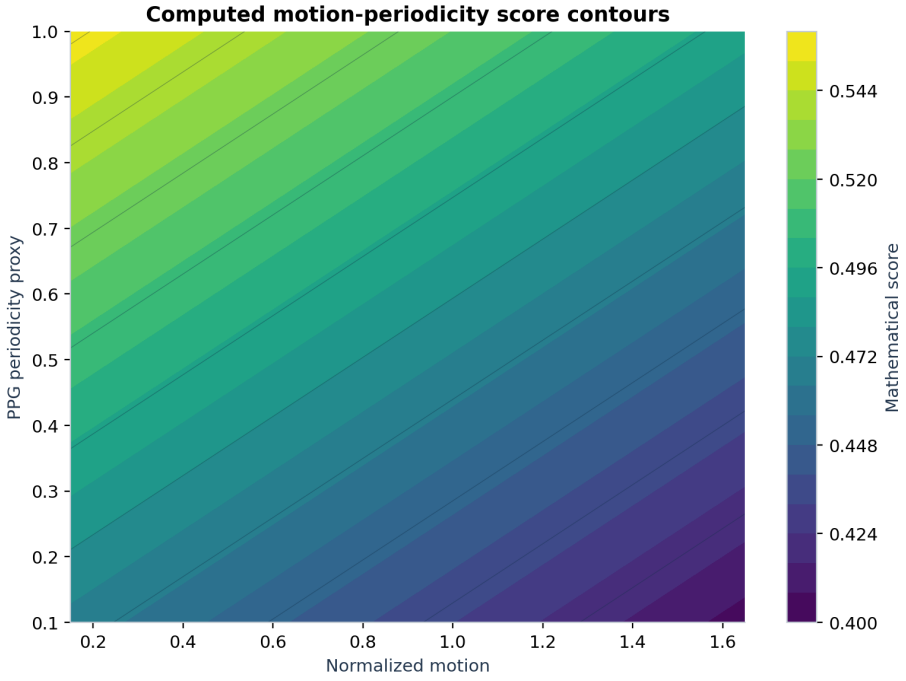


Figure 6. Contour map of the same 3,721 paceval-evaluated mathematical input combinations.

8. Reusable formulas and controlled revision

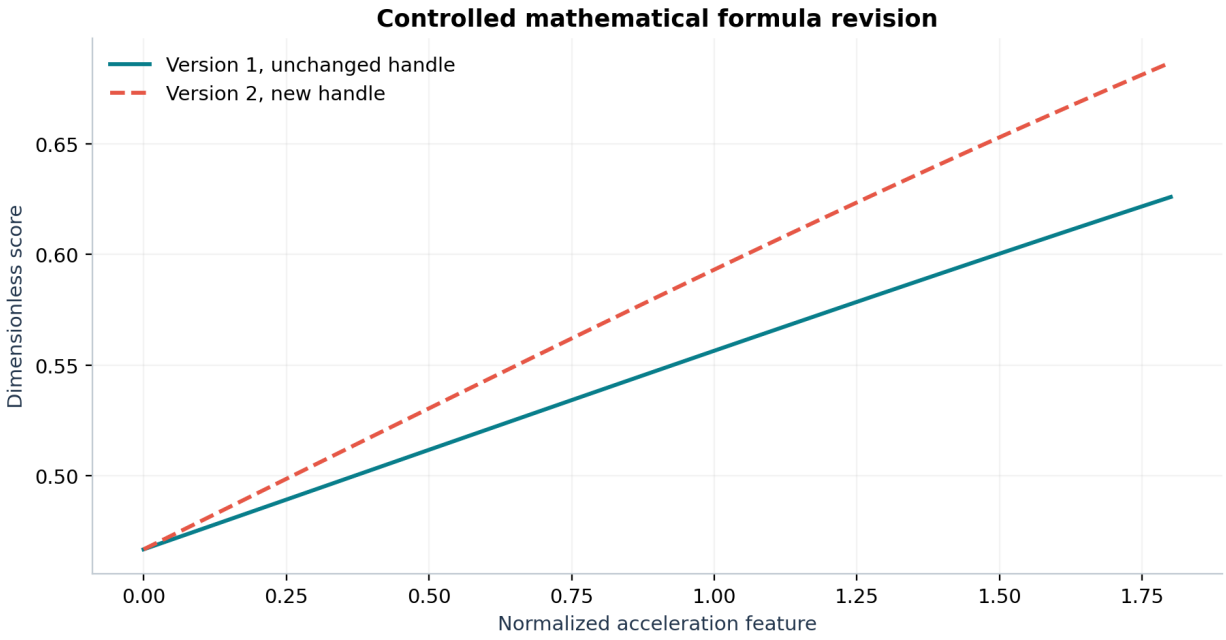


Figure 7. A newly created formula version changes the activity score while the earlier graph remains independently callable.

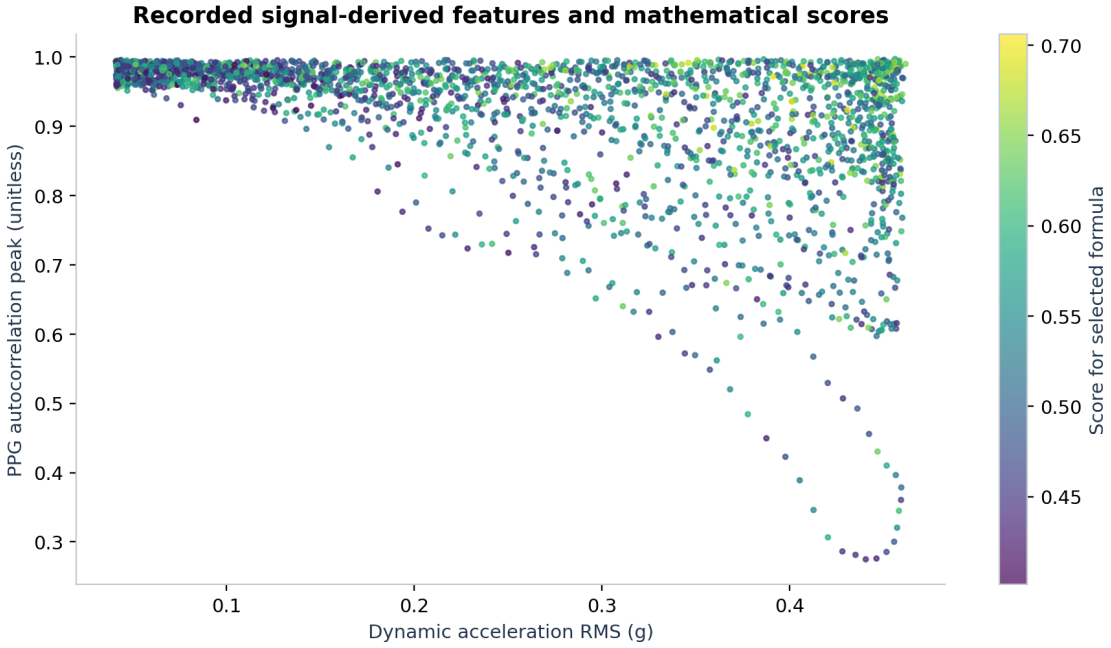


Figure 8. Signal-derived features colored by their assigned mathematical score. The synthetic data do not represent population statistics.

9. Scientific validation and interpretation

What was independently verified

The complete 10,000-window C++ replay passed an independently programmed mathematical reference for every score. Fifty different graphs were evaluated via the official batch function on 32 additional vectors each. A separate 80-decimal-digit Python implementation checked 500 exported scores. Four saved raw-trace windows yielded 24 independently reconstructed feature values.

Integration build and CTest	1 / 1 passed with the original paceval runtime
Independent decimal calculations	500 of 500 passed
Raw signal feature reconstructions	24 of 24 passed
Expected graph count	51 independently retrievable handles
Input contract checks	3 deliberate invalid inputs rejected

Why the results are relevant to a wearable pipeline

Reusable mathematical expression handles allow an application to maintain several explicit, auditable postprocessing equations independently of signal acquisition, detection or sensor-specific preprocessing. Version 2 was registered without invalidating version 1, allowing a controlled formula change and side-by-side numerical assessment.

Scientific and medical boundaries

The data contain no real participants, no collected optical or inertial recordings, no validated physiological target labels and no clinical study. The result does not demonstrate a diagnosis, validated activity recognition, sensor robustness, medical device eligibility, calibrated probabilities or any execution-time, power or battery advantage.

10. Reproducibility and external references

Reproduce with the original C/C++ SDK and external runtime

The ZIP includes src/wearable_scoring.cpp, original paceval_main.h/cpp SDK files, src/CMakeLists.txt, deterministic raw-signal logic, 51 serialized formula definitions, full score data, high-precision audits, eight figure scripts/output pairs, and this whitepaper generator.

Build: cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so; then cmake --build build -j; ctest --test-dir build --output-on-failure. Run ./build/wearable_scoring --windows 10000 --out examples, followed by the independent Python verification and figure scripts. All exact commands appear in README.md.

Source references used in the domain review

[1] Wearable ECG/PPG multimodal signal monitoring review (2026), Sensors 26(11):3477: https://www.mdpi.com/1424-8220/26/11/3477

[2] PPG signal quality assessment and reconstruction for heart-rate analysis (2025), Sensors 25(24):7556: https://pmc.ncbi.nlm.nih.gov/articles/PMC12736534/

[3] Reiss, Indlekofer & Schmidt (2019), PPG-DaLiA, UCI Machine Learning Repository: https://doi.org/10.24432/C53890

[4] PhysioNet Wrist PPG During Exercise v1.0.0: https://physionet.org/content/wrist/1.0.0/

[5] FDA General Wellness: Policy for Low Risk Devices, January 2026: https://www.fda.gov/regulatory-information/search-fda-guidance-documents/general-wellness-policy-low-risk-devices

[6] paceval original C/C++ interface and mathematical engine: https://github.com/paceval/paceval

Conclusion

Original paceval C/C++ execution successfully demonstrated 50 reusable mathematical wearable signal-score definitions and an immutable revision over deterministic synthetic PPG and motion features. The published 3D score surfaces and result figures follow directly from executed values. Transfer to actual wearables requires a separately validated sensor-processing and domain-specific study.